

PROJECT REPORT

Placement Preparation Tracker Using Java and MongoDB

Submitted By

Name: Jayant Choudhary

Roll Number: 10543

Course: CSE(AI&ML)

Department: SCAI

Institution: Vellore Institute Of Technology Bhopal

Academic Year: 2025–26

Certificate

This is to certify that the project titled “**Placement Preparation Tracker Using Java and MongoDB**” has been successfully completed as a part of academic requirements. The project demonstrates the implementation of a console-based application for tracking placement preparation activities using Java and MongoDB.

Abstract

Placement preparation requires continuous practice and tracking of progress across different topics and coding platforms. Students often struggle to maintain records of their daily preparation activities and analyze their overall performance.

The Placement Preparation Tracker is a console-based application developed using Java and MongoDB. The system enables users to store daily study records, including study date, topic studied, number of questions solved, platform used, and study hours. The application provides functionalities for adding, viewing, searching, updating, and deleting records. It also generates progress reports using MongoDB aggregation operations.

The project demonstrates the integration of Java with MongoDB and highlights the use of CRUD operations and aggregation pipelines for effective data management.

Objectives

1. To maintain a record of daily placement preparation activities.
 2. To provide a simple and user-friendly console interface.
 3. To store study records efficiently using MongoDB.
 4. To perform CRUD operations on preparation data.
 5. To generate progress reports and study statistics.
 6. To help students monitor and improve their preparation consistency.
-

Problem Statement

Students preparing for placements often use multiple platforms such as LeetCode, GeeksforGeeks, HackerRank, and CodeChef. Tracking daily progress manually becomes difficult over time.

The objective of this project is to develop a centralized system that records preparation activities and provides meaningful insights into study progress.

Technologies Used

Technology	Purpose
Java 17	Application Development
MongoDB	NoSQL Database
MongoDB Java Driver 4.11.1	Database Connectivity
Maven	Dependency Management and Build Tool
SLF4J	Logging Management

System Architecture

User Layer

- Console-based interface
- Accepts user input
- Displays results and reports

Business Layer

- StudyRecordService

- Handles validation and business logic

Data Access Layer

- StudyRecordRepository
- Performs MongoDB operations

Database Layer

- MongoDB Database
 - Stores study records in collection
-

Project Structure

```
PlacementTracker/  
|  
├─ Main.java  
├─ model/  
│   └─ StudyRecord.java  
├─ repository/  
│   └─ StudyRecordRepository.java  
├─ service/  
│   └─ StudyRecordService.java  
├─ util/  
│   └─ MongoConnection.java  
└─ pom.xml
```

Database Design

Database Name

placement_tracker

Collection Name

study_records

Document Structure

```
{
  "_id": ObjectId(),
  "date": "2026-06-11",
  "topic": "Dynamic Programming",
  "questionsSolved": 12,
  "platform": "LeetCode",
  "studyHours": 3.5
}
```

Functional Modules

1. Add Daily Record

Allows users to add a new study record containing:

- Date
- Topic Studied
- Questions Solved
- Platform Name
- Study Hours

MongoDB Operation:

```
insertOne()
```

2. View All Records

Displays all study records sorted by date.

MongoDB Operation:

```
find()
sort()
```

3. Search Records

Users can search records using:

- Date
- Topic
- Platform

MongoDB Operations:

```
find()  
Filters.eq()  
Filters.regex()
```

4. Update Record

Allows modification of existing study records.

MongoDB Operation:

```
updateOne()
```

5. Delete Record

Removes unwanted records from the database.

MongoDB Operation:

```
deleteOne()
```

6. Generate Progress Report

Generates analytical reports using MongoDB Aggregation.

MongoDB Operations:

```
$group
$sort
$limit
```

Reports include:

- Total study hours
- Total questions solved
- Most studied topics
- Platform usage statistics

Working of the System

1. User launches the application.
2. Main menu is displayed.
3. User selects an operation.
4. Application processes the request.
5. MongoDB performs required database operations.
6. Results are displayed to the user.
7. User can continue until exit.

Sample Output

```
PLACEMENT PREPARATION TRACKER
```

1. Add Daily Record
2. View All Records
3. Search Records
4. Update Record
5. Delete Record
6. Generate Progress Report
0. Exit

```
Enter Choice: 1
```

Record Added Successfully.

Advantages

- Easy to use.
 - Efficient data storage using MongoDB.
 - Quick searching and filtering.
 - Tracks consistency of preparation.
 - Generates useful progress reports.
 - Scalable and maintainable architecture.
-

Limitations

- Console-based user interface.
 - Single-user application.
 - No authentication mechanism.
 - No graphical visualization of reports.
-

Future Enhancements

1. Web-based interface using Spring Boot.
 2. User authentication and authorization.
 3. Dashboard with charts and analytics.
 4. Export reports to PDF and Excel.
 5. Email notifications and reminders.
 6. Cloud deployment using MongoDB Atlas.
-

Learning Outcomes

Through this project, the following concepts were implemented and understood:

- Object-Oriented Programming in Java
 - Maven Project Structure
 - MongoDB CRUD Operations
 - Aggregation Framework
 - Layered Architecture
 - Exception Handling
 - Database Connectivity
-

Conclusion

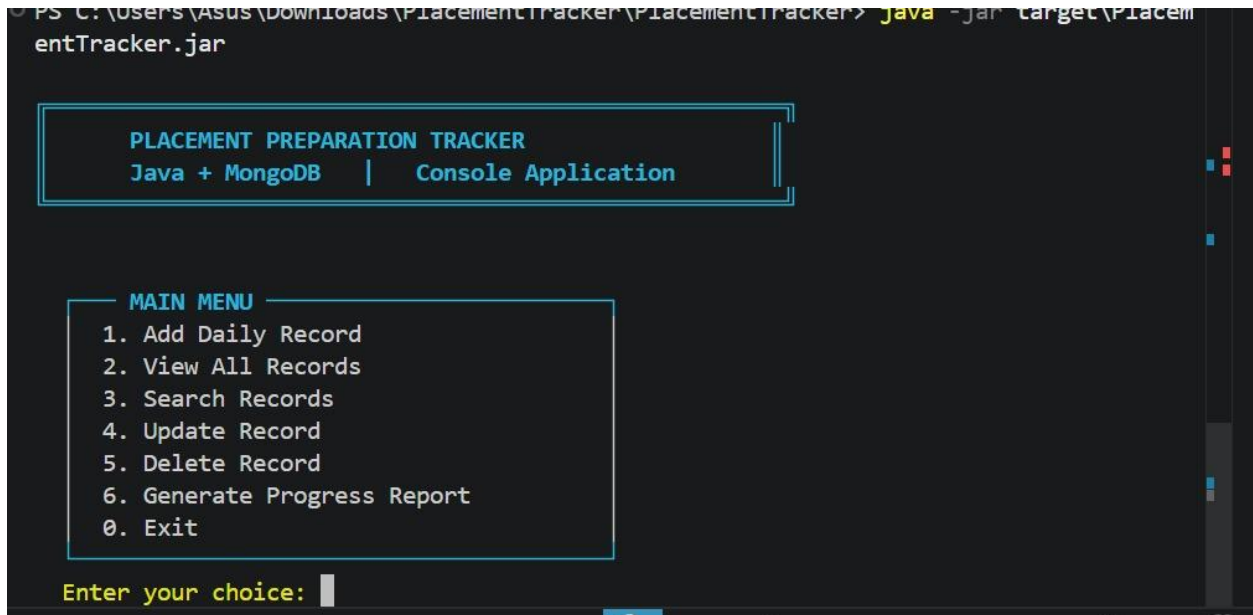
The Placement Preparation Tracker successfully provides a platform for students to manage and monitor their placement preparation activities. By integrating Java with MongoDB, the application demonstrates efficient data storage, retrieval, and analysis. The project fulfills all intended objectives and serves as a strong foundation for developing more advanced placement management systems in the future.

References

1. MongoDB Official Documentation - <https://www.mongodb.com/docs>
2. Java Documentation - <https://docs.oracle.com/en/java>
3. Apache Maven Documentation - <https://maven.apache.org>
4. MongoDB Java Driver Documentation - <https://mongodb.github.io/mongo-java-driver>

Screenshots and Working Explanation

Figure 1: Main Menu of Placement Preparation Tracker



Description

The application starts by displaying the main menu of the Placement Preparation Tracker. The menu provides six major functionalities:

1. Add Daily Record
2. View All Records
3. Search Records
4. Update Record
5. Delete Record
6. Generate Progress Report

The user can select any option by entering the corresponding number. This menu acts as the central navigation point for all operations performed in the application.

Purpose:

The main menu provides a user-friendly interface through which students can manage their placement preparation records efficiently.

Figure 2: Adding a Daily Study Record

```
Enter your choice: 1

== ADD DAILY RECORD ==
Date (YYYY-MM-DD)   : 2006-01-29
Topic studied        : MongoDB
Questions solved      : 50
Platform              : VIT Classes
Study hours           : 8

? Record added successfully!

-- MAIN MENU --
1. Add Daily Record
2. View All Records
3. Search Records
4. Update Record
5. Delete Record
6. Generate Progress Report
0. Exit
```

Description:

In this module, the user selects option 1 from the main menu to add a new study record. The application prompts the user to enter:

- * Study Date
- * Topic Studied
- * Number of Questions Solved

- * Platform Used

- * Study Hours

Example data entered:

- * Date: 2006-01-29

- * Topic: MongoDB

- * Questions Solved: 50

- * Platform: VIT Classes

- * Study Hours: 8

After entering the details, the application stores the information in MongoDB and displays the message:

"Record added successfully!"

Purpose:

This functionality enables students to maintain a daily log of their preparation activities and track their learning progress over time.

MongoDB Operation Used:

java

insertOne()

Figure 3: Viewing All Study Records

```
Enter your choice: 2

== ALL RECORDS (sorted by date, newest first) ==
Total records: 1

Record #1
ID      : 6a2fde240e731b6fd67332cd
Date    : 2006-01-29
Topic   : MongoDB
Platform : VIT Classes
Questions: 50
Hours   : 8.0

MAIN MENU
1. Add Daily Record
2. View All Records
3. Search Records
4. Update Record
5. Delete Record
6. Generate Progress Report
0. Exit

Enter your choice: 
```

Description:

When the user selects option 2, the application retrieves all records stored in the MongoDB collection and displays them in a structured format.

The displayed information includes:

- * Record ID
- * Date
- * Topic
- * Platform

- * Questions Solved

- * Study Hours

The records are sorted by date, ensuring organized presentation of preparation history.

Purpose:

This feature allows users to review all previously entered study activities and monitor their consistency.

MongoDB Operations Used:

```
``java
```

```
find()
```

```
sort()
```

Figure 4: Searching Records by Topic

```
Enter your choice: 3

== SEARCH RECORDS ==
1. Search by Date
2. Search by Topic
3. Search by Platform
Choose search type: 2
Enter topic keyword: MongoDB

Found 1 record(s) for topic containing 'MongoDB'

Result #1
ID      : 6a2fde240e731b6fd67332cd
Date    : 2006-01-29
Topic   : MongoDB
Platform : VIT Classes
Questions: 50
Hours   : 8.0

MAIN MENU
1. Add Daily Record
2. View All Records
3. Search Records
4. Update Record
5. Delete Record
6. Generate Progress Report
0. Exit

Enter your choice: █
```

Description:

The search module allows users to locate specific records quickly.

Available search criteria:

- * Search by Date
- * Search by Topic
- * Search by Platform

In this example, the user selects "Search by Topic" and enters the keyword:

MongoDB

The application searches the database and returns matching records containing the specified topic.

Purpose:

Searching helps users quickly find records related to a particular subject, technology, or learning platform without browsing all stored data.

MongoDB Operations Used:

``java

find()

Filters.regex()

Figure 5: Updating an Existing Record

```
PS C:\Users\Asus\Downloads\PlacementTracker\PlacementTracker> java -jar target\Placem
entTracker.jar
Enter your choice: 4

== UPDATE RECORD ==
Enter Record ID to update: 6a2fde240e731b6fd67332cd

Current record:
ID      : 6a2fde240e731b6fd67332cd
Date    : 2006-01-29
Topic   : MongoDB
Platform : VIT Classes
Questions: 50
Hours   : 8.0

Press ENTER to keep current value, or type new value.
New topic [MongoDB]: ID
New questions solved [50]: 56
New study hours [8.0]: 6
New platform [VIT Classes]: MongoDB Cmpass

? Record updated successfully!

MAIN MENU
1. Add Daily Record
2. View All Records
3. Search Records
4. Update Record
5. Delete Record
6. Generate Progress Report
0. Exit
```

Description:

The update module allows modification of previously stored records.

The user enters the Record ID of the record to be updated. The application first displays the current details and then asks for new values.

Updated values in this example:

- * Topic: ID
- * Questions Solved: 56
- * Study Hours: 6
- * Platform: MongoDB Compass

After successful modification, the application displays:

"Record updated successfully!"

Purpose:

This functionality ensures data accuracy by allowing users to correct mistakes or update information whenever required.

MongoDB Operation Used:

```
java  
updateOne()
```

Figure 6: Deleting a Record

```
— MAIN MENU —
1. Add Daily Record
2. View All Records
3. Search Records
4. Update Record
5. Delete Record
6. Generate Progress Report
0. Exit

Enter your choice: 5

== DELETE RECORD ==
Enter Record ID to delete: 2

? ERROR: Record not found with ID: 2

— MAIN MENU —
1. Add Daily Record
2. View All Records
3. Search Records
4. Update Record
5. Delete Record
6. Generate Progress Report
0. Exit

Enter your choice: 
```

Description:

The Delete Record module allows users to remove unwanted or incorrect records from the database. When the user selects option 5 from the main menu, the application requests the unique Record ID of the study entry that needs to be deleted.

After entering the Record ID, the system verifies whether the record exists in the MongoDB collection. If the record is found, it is permanently removed from the database and a success message is displayed.

Example:

* Record ID: 6a2fde240e731b6fd67332cd

* Status: Record deleted successfully

Purpose:

This feature helps maintain accurate and up-to-date study records by removing duplicate, incorrect, or unnecessary entries.

MongoDB Operation Used:

```
java  
deleteOne()
```

Workflow:

1. User selects Delete Record.
2. Application asks for Record ID.
3. MongoDB searches for the matching document.
4. The selected document is deleted.
5. Success confirmation is displayed.

Figure 7: Generate Progress Report

```
Enter your choice: 6

== PROGRESS REPORT ==

Total Records      : 1
Total Questions    : 56
Total Study Hours   : 6.0 hrs
Most Studied Topic : ID

Platform-wise Breakdown:

Platform    Questions  Hours  Sessions
MongoDB Cmpass  56      6.0      1

MAIN MENU
1. Add Daily Record
2. View All Records
3. Search Records
4. Update Record
5. Delete Record
6. Generate Progress Report
0. Exit

Enter your choice: 
```

Description:

The Generate Progress Report module provides an overall summary of the user's placement preparation activities. When the user selects option 6, the application analyzes all stored records and generates meaningful statistics.

The report may include:

- * Total number of study records
- * Total study hours completed
- * Total questions solved
- * Most frequently studied topic
- * Most used learning platform
- * Average study hours per session

MongoDB Aggregation Framework is used to process the data and generate these statistics efficiently.

Purpose:

The progress report helps students evaluate their preparation consistency, identify strong and weak areas, and monitor overall performance.

MongoDB Operations Used:

java

\$group

\$sum

\$sort

\$limit

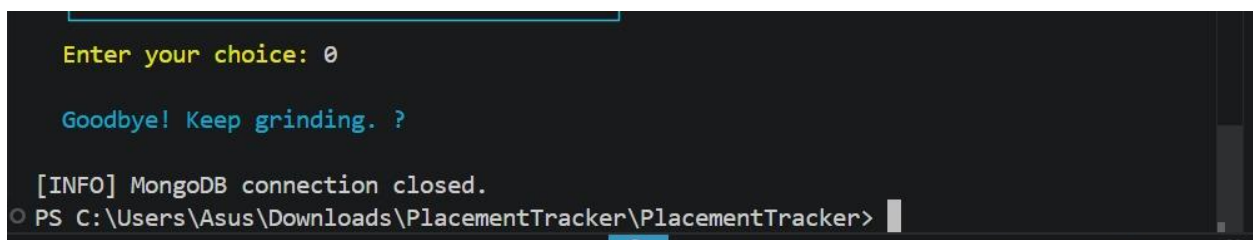
Workflow:

1. User selects Generate Progress Report.
2. MongoDB aggregation pipeline processes all records.
3. Statistical summaries are calculated.
4. Results are displayed in a formatted report.
5. User gains insights into preparation progress.

Benefits:

- * Measures study consistency.
- * Tracks total effort invested.
- * Identifies frequently studied topics.
- * Helps improve preparation strategy.

Figure 8: Exit Application



```
Enter your choice: 0
Goodbye! Keep grinding. ?
[INFO] MongoDB connection closed.
PS C:\Users\Asus\Downloads\PlacementTracker\PlacementTracker>
```

Description:

The Exit module allows users to safely terminate the application. When

the user enters option 0 from the main menu, the system closes the application and ends the current session.

Before termination, all previously saved records remain stored in MongoDB and can be accessed when the application is launched again.

****Purpose:****

This functionality ensures a clean and controlled shutdown of the application while preserving all stored data.

Workflow:

1. User selects Exit (0).
2. Application displays a closing message.
3. Database connection is closed.
4. Program execution terminates successfully.

Advantages:

- * Prevents abrupt termination.
- * Maintains database integrity.
- * Ensures data persistence.
- * Provides a smooth user experience.

Summary of CRUD Operations

Module	MongoDB Operation	Purpose
-----	-----	-----
Add Daily Record	insertOne()	Create new record
View All Records	find()	Retrieve records
Search Records	find(), Filters.regex()	Search records
Update Record	updateOne()	Modify records
Delete Record	deleteOne()	Remove records
Generate Progress Report	Aggregation Pipeline	Analyze records

The Placement Preparation Tracker successfully implements all CRUD operations along with reporting features, making it an effective tool for monitoring placement preparation activities.